Tarea. Manipulación y consultas avanzadas en un sistema NoSQL

Historia Clínica Electrónica Básica — Subsistema de Hospitalización
Gobernación de Antioquia · Modelo documental en MongoDB

Asignatura	Bases de Datos II (G101) — Tecnología en Desarrollo de Software
Profesora	Oralia Cortés Grajales
Actividad	AE3 · Unidad 3 · Informe técnico
Modalidad	Trabajo individual
Autor	Henry Taborda
SGBD	MongoDB Community 8.3.7 + mongosh (modelo documental híbrido)
Base de datos	hce_hospitalizacion_u3
Repositorio	https://github.com/HenryStark866/bd2-ae3-u3-hce-hospitalizacion
Video	Pendiente: agregar enlace en config.json
Fecha	8 de noviembre de 2026

Contenido: 1. Modelo documental · 2. Datos simulados · 3. Operaciones CRUD · 4. Consultas avanzadas (A1–A7) y optimización (O1) · 5. Integración de IA · 6. Reflexión ética · 7. Distribución del trabajo, repositorio y video · 8. Conclusiones.

Nota: todos los datos de este trabajo son **ficticios**. Las salidas que se muestran son copias exactas de la ejecución real de los scripts sobre MongoDB (carpeta `resultados/`).

1. Descripción del modelo documental

1.1 De la Unidad 2 a la Unidad 3

En la Unidad 2 se construyó un modelo **documental híbrido** en MongoDB: la hospitalización es el documento raíz y **embebe** lo que nace y muere con ella (diagnósticos, evolución, signos vitales, notas de enfermería, órdenes de medicamentos y alta), mientras que paciente, profesionales y catálogos se **referencian** porque se reutilizan entre muchas hospitalizaciones. Para este reto ese modelo no se reemplaza: se **amplía** con lo que exigen las operaciones CRUD complejas y el análisis clínico.

Cambio	Para qué se necesita
Colección `camas` (30 camas en 4 servicios, estado y hospitalización que la ocupa)	La admisión y el alta cambian dos colecciones; permite medir ocupación (A3).
Colección `auditoria` (solo se agrega, nunca se modifica)	Trazabilidad: cada operación CRUD deja quién, qué, cuándo y por qué.
Subdocumento `eliminado {fecha, por, motivo}` en pacientes y hospitalizaciones	Borrado lógico: la historia clínica no se destruye (Res. 1995/1999).
`familia_alergia` en el catálogo de medicamentos	Cruzar alergias del paciente con lo que se le ordena (A6).
`contacto.subregion` del paciente y `notificacion_sivigila` en la hospitalización	Análisis epidemiológico territorial (A1) y vigilancia en salud pública (U5).
De 7 a 35 hospitalizaciones generadas desde perfiles clínicos	Volumen suficiente para que las agregaciones revelen patrones (reingresos, estancias).

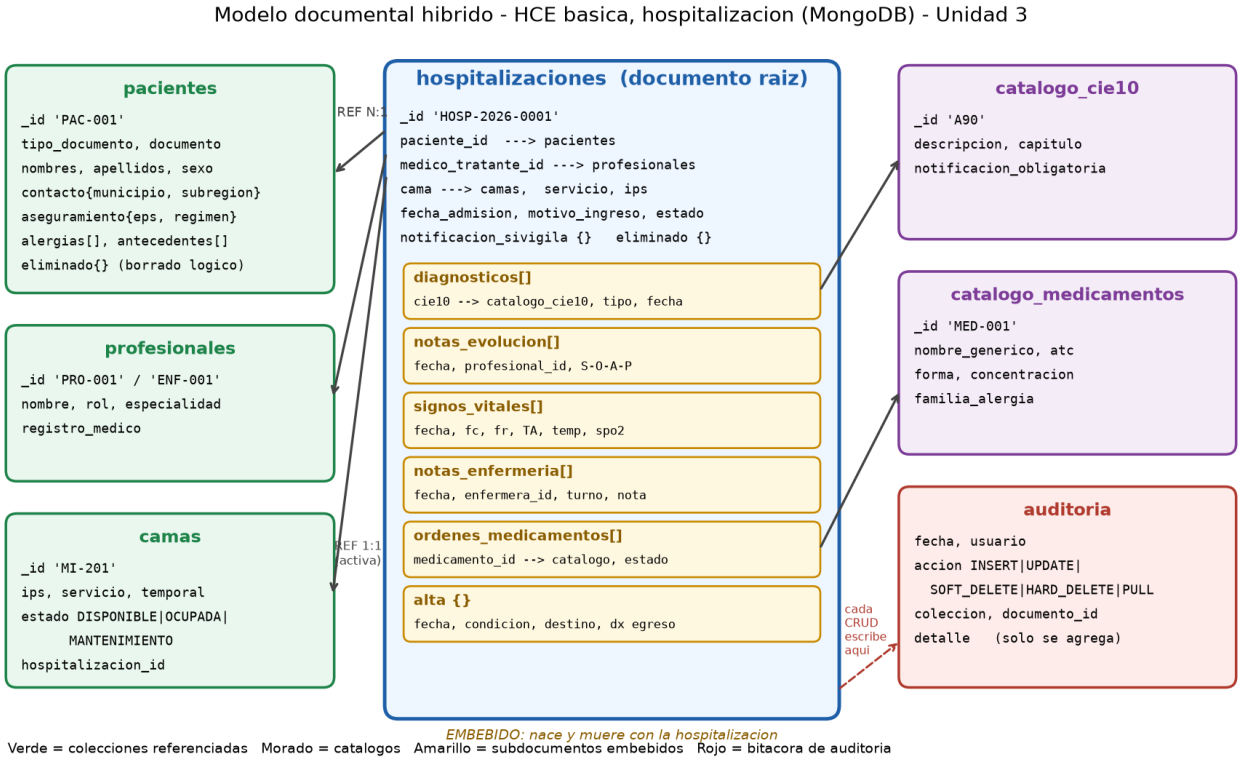


Figura 1. Modelo documental híbrido de la Unidad 3: núcleo embebido (amarillo), colecciones referenciadas (verde), catálogos (morado) y bitácora de auditoría (rojo).

1.2 Embebido vs. referenciado (decisiones)

Elemento	Estrategia	Justificación
Diagnósticos, evolución, signos vitales, enfermería, órdenes, alta	Embebido (arrays / objeto)	Se leen siempre con la hospitalización; su volumen por ingreso es acotado (~3 tomas de signos/día). Una sola lectura trae el episodio completo.
Paciente	Referenciado	Un paciente tiene varias hospitalizaciones (hay 4 reingresos). Los datos de identificación quedan aparte y se pueden proteger o seudonimizar por separado.
Profesionales y catálogos (CIE-10, medicamentos)	Referenciado	Datos maestros compartidos: se corrigen una vez. Se copia la descripción en el documento para lectura rápida (desnormalización controlada).
Camas	Referenciado (1:1 mientras está activa)	Su estado cambia con independencia del episodio y se consulta por servicio para gestionar capacidad.
Auditoría	Colección propia	Si se embebiera, quien edita la historia podría alterar su propio rastro.

1.3 Validación e índices

Cada colección tiene un validador `$jsonSchema` (tipos, enumeraciones, patrones, campos obligatorios y rangos fisiológicos: FC 20–250, SpO_2 50–100, temperatura 30–45 °C). Índices creados: `ux_documento` (único por tipo y número de documento), `ix_territorio`, `ix_paciente_fecha`, `ix_diagnostico` (multikey sobre `diagnosticos.cie10`), `ix_estado_servicio`, `ix_medicamento`, `ix_cama_servicio_estado` e `ix_auditoria_doc`.

```
// ---------- Camas ----------
hce.createCollection("camas", { validator: { $jsonSchema: {
  bsonType: "object", required: ["_id", "ips", "servicio", "estado"],
  properties: { _id: { bsonType: "string" }, ips: { bsonType: "string" },
    servicio: { enum: ["MEDICINA_INTERNA", "PEDIATRIA", "CIRUGIA", "GINECOOBSTETRICIA", "UCI"] },
    estado: { enum: ["DISPONIBLE", "OCUPADA", "MANTENIMIENTO"] },
    hospitalizacion_id: { bsonType: ["string", "null"], description: "REF -> hospitalizaciones._id si esta OCUPADA" },
    temporal: { bsonType: "bool", description: "cama de contingencia (expansion por brote)" } } } } });
// ---------- Auditoria (append-only) ----------
hce.createCollection("auditoria", { validator: { $jsonSchema: {
  bsonType: "object", required: ["fecha", "usuario", "accion", "coleccion", "documento_id"],
  properties: { fecha: { bsonType: "date" }, usuario: { bsonType: "string" },
    accion: { enum: ["INSERT", "UPDATE", "SOFT_DELETE", "HARD_DELETE", "PULL"] },
    coleccion: { bsonType: "string" }, documento_id: { bsonType: "string" },
    detalle: { bsonType: "string" } } } } });
```

Limitación encontrada: se intentó un índice parcial que indexara solo registros vigentes (`{eliminado: {$exists: false}}`); MongoDB lo rechaza porque los índices parciales no admiten negaciones. Se dejó un índice normal y la explicación en el script. Alternativa para producción: un booleano `vigente: true`, que sí admite índice parcial.

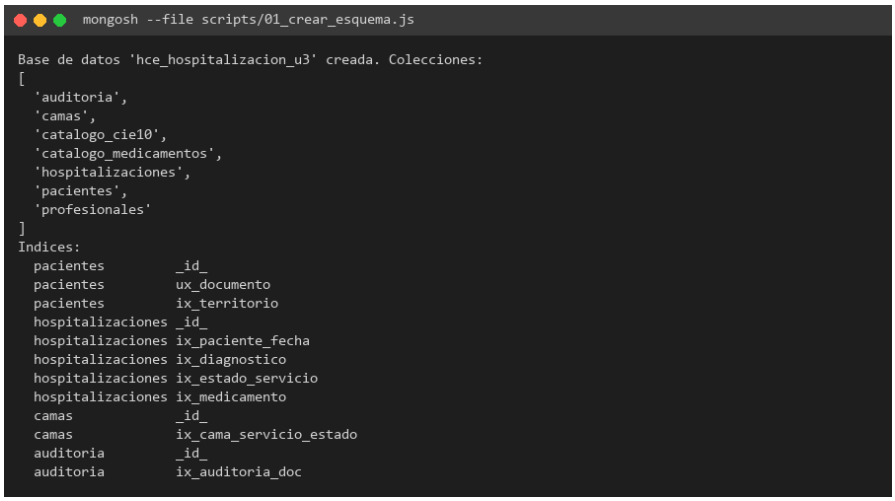


Figura 2. Ejecución de 01_crear_esquema.js: siete colecciones y sus índices.

2. Datos simulados

El script `02_carga_datos.js` carga 13 códigos CIE-10, 17 medicamentos (con código ATC y familia de alergia), 9 profesionales, 24 pacientes de 6 subregiones de Antioquia y 30 camas. Las 35 hospitalizaciones no se escribieron una por una: se generan desde **perfiles clínicos** (neumonía, EPOC, dengue, dengue grave, falla cardiaca, infarto, cetoacidosis diabética, apendicitis, etc.), cada uno con su servicio, medicamentos y la trayectoria de signos vitales desde el ingreso hasta el alta. Así los datos son coherentes: un paciente con neumonía ingresa con fiebre y SpO_2 baja y mejora; el dengue se concentra en Urabá y Bajo Cauca; los menores de 15 años van a Pediatría con dosis por peso.

```
const PERFILES = {
  NAC: { cie: "J18.9", servicio: "MEDICINA_INTERNA", via: "URGENCIAS", motivo: "Fiebre, tos productiva y disnea de 3
      días de evolución",
    enf: [106, 26, 126, 76, 38.7, 89],
    meds: [["MED-001", "1 g", "IV", "cada 12 horas", 2], ["MED-002", "500 mg", "VO", "cada 24 horas", 2], ["MED-010",
      "500 mg", "VO", "cada 8 horas si fiebre", 2]],
    evol: [["Disnea y fiebre.", "Crépitos basales, Rx con consolidación.", "Neumonía adquirida en comunidad CURB-65 =
      2.", "Oxígeno, antibiótico IV, hidratación."],
           ["Menos disnea, persiste tos.", "Afebril 24 h, SpO2 94 % con cánula.", "Buena respuesta al tratamiento.",
      "Continuar antibiótico, destete de oxígeno."],
           ["Asintomático respiratorio.", "SpO2 96 % al aire ambiente.", "Resolución clínica.", "Paso a vía oral y
      egreso."]] },
  ...  (12 perfiles en total; ver archivo completo)
```

```
mongosh --file scripts/02_carga_datos.js

Carga completada:
  catalogo_cie10          13
  catalogo_medicamentos   17
  profesionales           9
  pacientes               24
  camas                   30
  hospitalizaciones       35
  auditoria               0
  -> 7 hospitalizaciones activas, 491 tomas de signos vitales, 104 notas de evolucion, 86 ordenes, 114 diagnosticos
  -> camas ocupadas: 7 / 30
```

Figura 3. Resultado de la carga: 491 tomas de signos vitales, 104 notas de evolución, 86 órdenes y 114 diagnósticos embebidos.

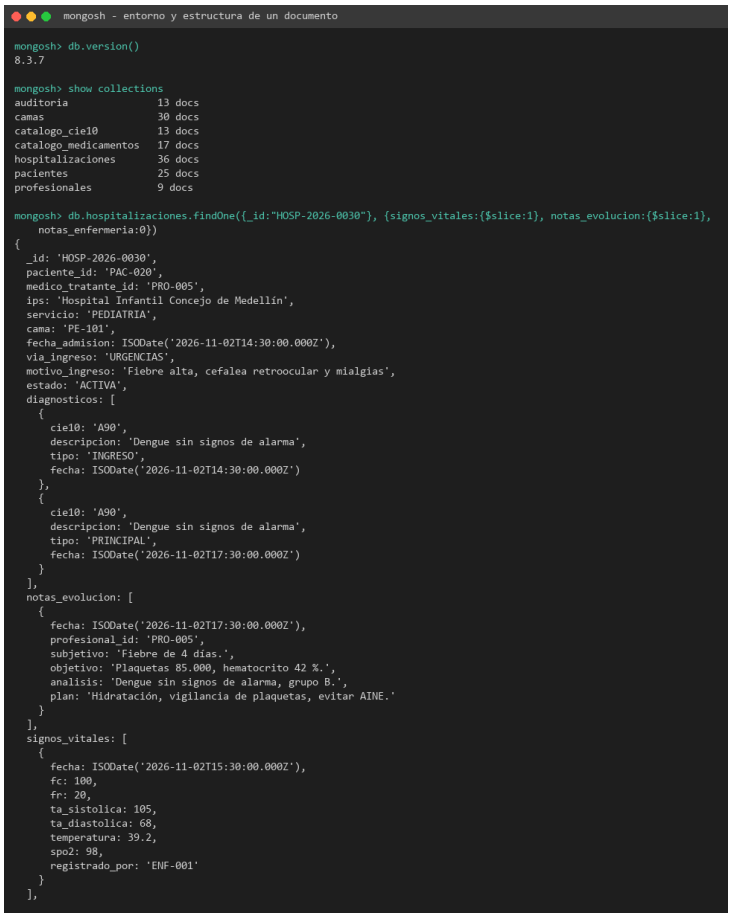


Figura 4. Servidor MongoDB 8.3.7, colecciones después de ejecutar todos los scripts y estructura de un documento de hospitalización (proyección con $slice).

3. Operaciones CRUD

El script `03_crud.js` ejecuta 14 operaciones sobre la base cargada. Cada operación que modifica datos escribe un registro en `auditoria`. El servidor es *standalone* (sin réplica), por lo que no hay transacciones multidocumento; donde dos colecciones cambian juntas (cama y hospitalización) se usa `findOneAndUpdate`, que es atómico por documento, y un orden de escritura que no deja estados huérfanos.

Op.	Operación MQL	Estructura manipulada	Qué demuestra
C1	insertOne	Paciente con subdocumentos contacto y aseguramiento	Inserción validada por $jsonSchema
C2	findOneAndUpdate + insertOne	Cama (estado) + hospitalización completa	Admisión: reserva atómica de cama libre
C3	insertOne (inválido)	Paciente con 5 errores	El validador rechaza (código 121) y dice qué regla falló
R1	find + filtros anidados + $regex	diagnosticos.cie10 dentro del array	Pacientes activos de M. Interna con diabetes
R2	findOne + $slice: -2	signos_vitales[]	Leer solo las últimas tomas sin traer todo el array
U1	$set anidado + $addToSet	contacto.telefono, contacto.acudiente, alergias[]	Campo simple, subdocumento y array sin duplicados
U2	$push + $each	notas_evolucion[], diagnosticos[], ordenes[]	Tres arrays embebidos en una sola operación
U3	$set con arrayFilters $[o]	Solo las órdenes que cumplen el filtro	Suspender el AINE del paciente alérgico
U4	Operador posicional $	El diagnóstico tipo INGRESO	Corregir un elemento identificado por la consulta
U5	updateMany con pipeline	notificacion_sivigila{} calculado	Notificar a SIVIGILA los 8 casos de dengue
U6	$set + $[] + $push + findOneAndUpdate	alta{}, todas las órdenes, cama	Alta: cierra órdenes y libera la cama
D1	updateOne (eliminado{})	Paciente duplicado	Borrado lógico: se oculta pero se conserva
D2	$pull	Un elemento de signos_vitales[]	Retirar una toma registrada en la historia equivocada
D3	deleteOne / deleteMany	Duplicado sin historia, camas temporales	Borrado definitivo solo donde es legítimo

3.1 Create

La admisión (C2) reserva la primera cama disponible de Medicina Interna y, en el mismo paso, la marca ocupada: si dos admisiones llegaran a la vez, `findOneAndUpdate` impide que ambas tomen la misma cama.

```
titulo("C2. Admision: reservar cama libre de forma atomica (findOneAndUpdate) + insertOne");
const NUEVA = "HOSP-2026-0036";
const cama = hce.camas.findOneAndUpdate(
  { servicio: "MEDICINA_INTERNA", estado: "DISPONIBLE" },
  { $set: { estado: "OCUPADA", hospitalizacion_id: NUEVA } },
  { sort: { _id: 1 }, returnDocument: "after" });
print("Cama asignada: " + cama._id + " -> " + cama.estado);
const adm = D("2026-11-08T07:30:00");
hce.hospitalizaciones.insertOne({
  _id: NUEVA, paciente_id: "PAC-025", medico_tratante_id: "PRO-002", ips: cama.ips, servicio: "MEDICINA_INTERNA",
  cama: cama._id, fecha_admision: adm, via_ingreso: "REMISION",
  motivo_ingreso: "Fiebre de 5 días, dolor abdominal y vómito (remitido de Necoclí)", estado: "ACTIVA",
  diagnosticos: [{ cie10: "A90", descripcion: "Dengue sin signos de alarma", tipo: "INGRESO", fecha: adm }],
  notas_evolucion: [], notas_enfermeria: [],
  signos_vitales: [{ fecha: adm, fc: NumberInt(108), fr: NumberInt(22), ta_sistolica: NumberInt(104), ta_diastolica:
```

```
      NumberInt(66),
                    temperatura: Double(38.8), spo2: NumberInt(97), registrado_por: "ENF-003" }],
  ordenes_medicamentos: [], alta: null });
auditar("INSERT", "hospitalizaciones", NUEVA, "Admision por remision, cama " + cama._id);
print("Hospitalizacion creada: " + NUEVA);
```

```
mongosh --file scripts/03_crud.js

==================== C1. insertOne: nuevo paciente (validado por $jsonSchema) ====================
insertedId: PAC-025
==================== C2. Admision: reservar cama libre de forma atomica (findOneAndUpdate) + insertOne ====================
Cama asignada: MI-206 -> OCUPADA
Hospitalizacion creada: HOSP-2026-0036
==================== C3. insertOne invalido: el validador lo rechaza ====================
Rechazado -> codigo 121 (DocumentValidationFailure). Reglas incumplidas:
  - documento  (pattern)
  - fecha_nacimiento  (bsonType)
  - sexo  (enum)
  - contacto  (required)
  - aseguramiento  (required)
```

Figura 5. C1–C3: paciente insertado, cama MI-206 asignada, hospitalización creada y documento inválido rechazado con el detalle de cada regla incumplida.

3.2 Read

```
titulo("R1. find con filtros anidados: pacientes activos de Medicina Interna con diabetes (E11.*)");
hce.hospitalizaciones.find(
  { ...vigente, estado: "ACTIVA", servicio: "MEDICINA_INTERNA", "diagnosticos.cie10": { $regex: "^E11" } },
  { _id: 1, paciente_id: 1, cama: 1, "diagnosticos.cie10": 1, "diagnosticos.tipo": 1 }).sort({ _id: 1 })
  .forEach(d => print(d._id + "  " + d.paciente_id + "  cama " + d.cama + "  dx: " +
    d.diagnosticos.map(x => x.cie10 + "/" + x.tipo).join(", ")));

titulo("R2. findOne con $slice: ultimas 2 tomas de signos vitales de HOSP-2026-0031");
printjson(hce.hospitalizaciones.findOne({ _id: "HOSP-2026-0031" },
  { _id: 0, paciente_id: 1, servicio: 1, signos_vitales: { $slice: -2 } }));
```

```
mongosh --file scripts/03_crud.js

==================== R1. find con filtros anidados: pacientes activos de Medicina Interna con diabetes (E11.*)
    ====================
HOSP-2026-0029  PAC-022  cama MI-201  dx: J18.9/INGRESO, J18.9/PRINCIPAL, E11.9/RELACIONADO
HOSP-2026-0033  PAC-006  cama MI-204  dx: N39.0/INGRESO, N39.0/PRINCIPAL, E11.9/RELACIONADO
==================== R2. findOne con $slice: ultimas 2 tomas de signos vitales de HOSP-2026-0031 ====================
{
  paciente_id: 'PAC-007',
  servicio: 'MEDICINA_INTERNA',
  signos_vitales: [
    {
      fecha: ISODate('2026-11-07T23:40:00.000Z'),
      fc: 119,
      fr: 28,
      ta_sistolica: 170,
      ta_diastolica: 98,
      temperatura: 36.9,
      spo2: 89,
      registrado_por: 'ENF-003'
    },
    {
      fecha: ISODate('2026-11-08T07:40:00.000Z'),
      fc: 116,
      fr: 28,
      ta_sistolica: 165,
      ta_diastolica: 98,
      temperatura: 36.9,
      spo2: 89,
      registrado_por: 'ENF-001'
    }
  ]
}
```

Figura 6. R1 encuentra dos pacientes hospitalizados con diabetes como diagnóstico relacionado; R2 trae solo las dos últimas tomas de signos vitales.

3.3 Update

Las actualizaciones trabajan sobre la estructura compleja del documento. La más representativa es **U3**: el paciente nuevo es alérgico a los AINE y tiene dengue (los AINE están contraindicados por riesgo de sangrado), pero en U2 se le ordenó ibuprofeno. Con `arrayFilters` se suspende *solo* la orden conflictiva, dejando el acetaminofén activo, y se registra el motivo.

```
titulo("U3. arrayFilters: suspender ordenes que chocan con la alergia del paciente");
// El ibuprofeno (AINE) esta contraindicado en dengue y el paciente es alergico a AINEs.
// $[o] actualiza SOLO los elementos del arreglo que cumplen el filtro.
r = hce.hospitalizaciones.updateOne({ _id: NUEVA },
  { $set: { "ordenes_medicamentos.$[o].estado": "SUSPENDIDA", "ordenes_medicamentos.$[o].fecha_fin":
      D("2026-11-08T10:20:00"),
            "ordenes_medicamentos.$[o].motivo_suspension": "Alergia a AINEs y contraindicación en dengue" } },
  { arrayFilters: [{ "o.medicamento_id": { $in: ["MED-007", "MED-016"] }, "o.estado": "ACTIVA" }] });
auditar("UPDATE", "hospitalizaciones", NUEVA, "Suspension de AINE por alergia");
hce.hospitalizaciones.findOne({ _id: NUEVA }).ordenes_medicamentos
  .forEach(o => print("  " + o.nombre.padEnd(14) + o.estado.padEnd(12) + (o.motivo_suspension || "")));
```

U5 usa una actualización con *pipeline*: el número de ficha se calcula a partir del propio documento (código de evento del INS: 210 dengue, 220 dengue grave) y la lista de diagnósticos notificables se lee del catálogo, no se escribe a mano.

```
titulo("U5. updateMany: marcar notificacion SIVIGILA en todos los casos de dengue (A90/A91)");
const notificables = hce.catalogo_cie10.distinct("_id", { notificacion_obligatoria: true });
r = hce.hospitalizaciones.updateMany(
  { ...vigente, "diagnosticos": { $elemMatch: { cie10: { $in: notificables }, tipo: { $in: ["INGRESO", "PRINCIPAL"] }
      } },
    "notificacion_sivigila.notificado": { $ne: true } },
  [ { $set: { notificacion_sivigila: { notificado: true, fecha: "$$NOW",
      // codigo de evento INS: 210 = dengue, 220 = dengue grave (A91)
      ficha: { $concat: [{ $cond: [{ $in: ["A91", "$diagnosticos.cie10"] }, "220-", "210-"] },
                         { $substrCP: ["$_id", 10, 4] }] } } } } ]);   // update con pipeline
```

```
auditar("UPDATE", "hospitalizaciones", "*", "Notificacion SIVIGILA de " + r.modifiedCount + " casos de dengue");
print("codigos notificables: " + notificables.join(", ") + " | matched " + r.matchedCount + ", modified " +
      r.modifiedCount);
```

U6 registra el alta: llena el subdocumento `alta`, finaliza todas las órdenes con el operador `$[]`, agrega el diagnóstico de egreso y libera la cama.

```
titulo("U6. Alta: $set del subdocumento alta + $push dx de egreso + liberar la cama");
const tAlta = D("2026-11-08T11:00:00");
r = hce.hospitalizaciones.updateOne({ _id: "HOSP-2026-0034", estado: "ACTIVA" }, {
  $set: { estado: "EGRESADA", alta: { fecha: tAlta, condicion: "MEJORADO", destino: "Domicilio", diagnostico_egreso:
      "K35.8",
          recomendaciones: "Control de herida en 7 días." },
          "ordenes_medicamentos.$[].estado": "FINALIZADA", "ordenes_medicamentos.$[].fecha_fin": tAlta },
  $push: { diagnosticos: { cie10: "K35.8", descripcion: "Apendicitis aguda, otra y la no especificada", tipo:
      "EGRESO", fecha: tAlta } } });
const libera = hce.camas.findOneAndUpdate({ hospitalizacion_id: "HOSP-2026-0034" },
  { $set: { estado: "DISPONIBLE", hospitalizacion_id: null } }, { returnDocument: "after" });
auditar("UPDATE", "hospitalizaciones", "HOSP-2026-0034", "Egreso y liberacion de cama " + libera._id);
print("hospitalizacion modified " + r.modifiedCount + " | cama " + libera._id + " -> " + libera.estado);
```

```
mongosh --file scripts/03_crud.js

==================== U1. Campo simple y anidado: telefono (contacto.telefono) y $addToSet de alergia ====================
matched 1, modified 1
{
  contacto: {
    municipio: 'Necoclí',
    subregion: 'Uraba',
    telefono: '3125550025',
    acudiente: {
      nombre: 'Marleny Patiño',
      parentesco: 'Madre'
    }
  },
  alergias: [
    'AINEs'
  ]
}
==================== U2. $push en arreglos embebidos: nota de evolucion, diagnostico y ordenes (con $each) ====================
modified 1 -> ordenes ahora: Acetaminofén (ACTIVA), Ibuprofeno (ACTIVA)
==================== U3. arrayFilters: suspender ordenes que chocan con la alergia del paciente ====================
  Acetaminofén  ACTIVA
  Ibuprofeno    SUSPENDIDA  Alergia a AINEs y contraindicación en dengue
```

Figura 7. U1–U3: actualización anidada, $push con $each y suspensión selectiva con arrayFilters.

```
mongosh --file scripts/03_crud.js

==================== U4. Operador posicional $: corregir el diagnostico de INGRESO ====================
modified 1 -> Infección de vías urinarias (pielonefritis aguda)
==================== U5. updateMany: marcar notificacion SIVIGILA en todos los casos de dengue (A90/A91) ====================
codigos notificables: A90, A91 | matched 8, modified 8
==================== U6. Alta: $set del subdocumento alta + $push dx de egreso + liberar la cama ====================
hospitalizacion modified 1 | cama CI-301 -> DISPONIBLE
```

Figura 8. U4–U6: corrección posicional, notificación masiva (8 casos) y alta con liberación de cama.

3.4 Delete: lógico, $pull y definitivo

La historia clínica tiene obligación de conservación (Res. 1995/1999, modificada por la Res. 839/2017), por eso el borrado por defecto es **lógico**: el documento recibe `eliminado {fecha, por, motivo}` y todas las consultas lo excluyen con el filtro `vigente`. El borrado **definitivo** se reserva para lo que no es historia: un registro duplicado *sin hospitalizaciones* (el script lo verifica antes de borrar) y camas de contingencia ya desmontadas.

```
titulo("D1. Borrado LOGICO (soft delete): registro duplicado de paciente");
hce.pacientes.insertOne({ _id: "PAC-026", tipo_documento: "TI", documento: "1001250025", nombres: "Jhon Fredy",
      apellidos: "Holguín Patiño",
  fecha_nacimiento: D("1984-02-17T00:00:00"), sexo: "M", contacto: { municipio: "Necoclí", subregion: "Uraba" },
  aseguramiento: { eps: "Coosalud", regimen: "SUBSIDIADO" }, alergias: [], antecedentes: [] });
```

```
auditar("INSERT", "pacientes", "PAC-026", "Registro creado por error en admision (TI en vez de CC)");
r = hce.pacientes.updateOne({ _id: "PAC-026", ...vigente },
  { $set: { eliminado: { fecha: new Date(), por: USUARIO, motivo: "Duplicado de PAC-025 (mismo documento con tipo
      TI)" } } });
auditar("SOFT_DELETE", "pacientes", "PAC-026", "Duplicado de PAC-025");
print("marcado eliminado: " + r.modifiedCount);
print("pacientes totales: " + hce.pacientes.countDocuments() + " | vigentes: " +
      hce.pacientes.countDocuments(vigente));
titulo("D3. Borrado DEFINITIVO: deleteOne del duplicado (sin contenido clinico) y deleteMany de camas temporales");
// La historia clinica no se borra (Res. 1995/1999). Solo se elimina fisicamente lo que NO es historia:
// un registro duplicado sin ninguna hospitalizacion asociada, y camas de contingencia ya desmontadas.
const tieneHistoria = hce.hospitalizaciones.countDocuments({ paciente_id: "PAC-026" }) > 0;
if (!tieneHistoria) {
  r = hce.pacientes.deleteOne({ _id: "PAC-026", "eliminado.motivo": { $exists: true } });
  auditar("HARD_DELETE", "pacientes", "PAC-026", "Duplicado sin hospitalizaciones, eliminado definitivamente");
  print("PAC-026 tenia hospitalizaciones: " + tieneHistoria + " -> deletedCount " + r.deletedCount);
}
hce.camas.insertMany(["UCI-07", "UCI-08", "UCI-09"].map(id => ({ _id: id, ips: "Clínica Cardiovascular de Antioquia",
  servicio: "UCI", estado: "DISPONIBLE", hospitalizacion_id: null, temporal: true })));
print("camas con temporales: " + hce.camas.countDocuments());
r = hce.camas.deleteMany({ temporal: true, estado: "DISPONIBLE" });
auditar("HARD_DELETE", "camas", "UCI-07..09", "Fin de la contingencia por dengue");
print("deleteMany camas temporales: " + r.deletedCount + " | camas ahora: " + hce.camas.countDocuments());
```

```
mongosh --file scripts/03_crud.js

==================== D1. Borrado LOGICO (soft delete): registro duplicado de paciente ====================
marcado eliminado: 1
pacientes totales: 26 | vigentes: 25
==================== D2. $pull: retirar una toma de signos vitales registrada en la historia equivocada ====================
tomas antes: 2
tomas despues: 1
==================== D3. Borrado DEFINITIVO: deleteOne del duplicado (sin contenido clinico) y deleteMany de camas temporales
    ====================
PAC-026 tenia hospitalizaciones: false -> deletedCount 1
camas con temporales: 33
deleteMany camas temporales: 3 | camas ahora: 30
==================== Resumen: trazabilidad en la coleccion auditoria ====================
Registros de auditoria generados por este script: 13
  HARD_DELETE 2
  INSERT      3
  PULL        1
  SOFT_DELETE 1
  UPDATE      6
```

Figura 9. D1–D3 y resumen de auditoría: 13 registros (3 INSERT, 6 UPDATE, 1 SOFT_DELETE, 1 PULL, 2 HARD_DELETE).

4. Consultas avanzadas (Aggregation Framework)

Las consultas están en `04_consultas_avanzadas.js`. Cada una responde una pregunta que un hospital o la Secretaría de Salud se haría de verdad, usa entre 6 y 10 etapas (incluyendo `$match`, `$group`, `$project` y `$lookup`) y excluye los registros con borrado lógico. La fecha de corte del reporte es el 8 de noviembre de 2026.

#	Pregunta clínica	Operadores destacados	Responsable
A1	¿Dónde se concentra cada tipo de enfermedad en Antioquia?	$match, $unwind, $lookup, $group, $sort, $project	Henry Taborda
A2	¿Qué pacientes vuelven antes de 30 días tras el alta y por la misma causa?	$match, $setWindowFields, $shift, $set, $dateDiff, $filter, $lookup, $project, $sort	Henry Taborda
A3	¿Cuánta capacidad queda en cada servicio y quién ocupa cada cama?	$lookup, $group, $cond, $set, $sort, $project	Henry Taborda
A4	¿Qué antibióticos se usan, para qué diagnósticos y durante cuánto tiempo?	$match, $set, $unwind, $lookup, $regex, $group, $dateDiff, $sort, $project	Henry Taborda
A5	¿Qué paciente hospitalizado necesita valoración urgente ahora?	$match, $lookup, $set, $sortArray, $last, $switch, $sort, $project	Henry Taborda
A6	¿A quién se le ordenó un medicamento de una familia a la que es alérgico?	$match, $lookup, $unwind, $set, $map, $expr, $in, $project, $sort	Henry Taborda
A7	¿Qué diagnósticos consumen más días-cama y cuáles tienen peor desenlace?	$match, $set, $dateDiff, $group, $sum, $avg, $median, $max, $cond, $lookup, $sort, $project	Henry Taborda
O1	¿Qué tanto ayuda el orden de las etapas y el índice?	explain('executionStats')	Henry Taborda

A1. Perfil epidemiológico por subregión y capítulo CIE-10

Pregunta: ¿Dónde se concentra cada tipo de enfermedad en Antioquia?
Etapas: $match → $unwind → $match → $lookup (sub-pipeline que solo trae la subregión) → $lookup catálogo → $group → $sort → $group → $project
Responsable: Henry Taborda

```
titulo("A1. Perfil epidemiologico: hospitalizaciones por subregion y capitulo CIE-10");
hce.hospitalizaciones.aggregate([
  { $match: vigente },
  { $unwind: "$diagnosticos" },
  { $match: { "diagnosticos.tipo": "PRINCIPAL" } },
  { $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "_id", as: "p",
               pipeline: [{ $project: { _id: 0, subregion: "$contacto.subregion" } }] } },   // minimizacion: solo el
      territorio
  { $lookup: { from: "catalogo_cie10", localField: "diagnosticos.cie10", foreignField: "_id", as: "c" } },
  { $group: { _id: { subregion: { $first: "$p.subregion" }, capitulo: { $first: "$c.capitulo" } }, casos: { $sum: 1 }
      } },
  { $sort: { casos: -1 } },
  { $group: { _id: "$_id.subregion", total: { $sum: "$casos" },
              capitulos: { $push: { $concat: ["$_id.capitulo", "=", { $toString: "$casos" }] } } } },
  { $sort: { total: -1, _id: 1 } },
  { $project: { _id: 0, subregion: "$_id", total: "$total", capitulos: "$capitulos" } }
]).forEach(fila);
```

```
mongosh --file scripts/04_consultas_avanzadas.js

==================== A1. Perfil epidemiologico: hospitalizaciones por subregion y capitulo CIE-10 ====================
subregion: Valle de Aburra | total: 20 | capitulos: [Respiratorio=9, Circulatorio=6, Infecciosas=2, Digestivo=1, Endocrino=1,
    Genitourinario=1]
subregion: Uraba | total: 9 | capitulos: [Infecciosas=6, Genitourinario=2, Endocrino=1]
subregion: Oriente | total: 3 | capitulos: [Digestivo=2, Respiratorio=1]
subregion: Bajo Cauca | total: 2 | capitulos: [Infecciosas=2]
subregion: Magdalena Medio | total: 1 | capitulos: [Infecciosas=1]
subregion: Occidente | total: 1 | capitulos: [Circulatorio=1]
```

Figura 10. Resultado real de A1.

Qué revela: El dengue (capítulo Infecciosas) se concentra en Urabá (6 de 9 casos), Bajo Cauca y Magdalena Medio; en el Valle de Aburrá predominan las enfermedades respiratorias (9 de 20) y circulatorias (6). Es la base de un reporte epidemiológico territorial. El $lookup trae solo la subregión del paciente, no su identidad (minimización).

A2. Reingresos hospitalarios a 30 días

Pregunta: ¿Qué pacientes vuelven antes de 30 días tras el alta y por la misma causa?
Etapas: $match → $setWindowFields ($shift por paciente) → $set ($dateDiff, $filter) → $match → $lookup → $project → $sort
Responsable: Henry Taborda

```
titulo("A2. Reingresos a 30 dias (ventana por paciente con $setWindowFields)");
hce.hospitalizaciones.aggregate([
  { $match: vigente },
  { $setWindowFields: { partitionBy: "$paciente_id", sortBy: { fecha_admision: 1 }, output: {
      alta_anterior: { $shift: { output: "$alta.fecha", by: -1 } },
      dx_anterior: { $shift: { output: "$alta.diagnostico_egreso", by: -1 } },
      hosp_anterior: { $shift: { output: "$_id", by: -1 } } } } },
  { $set: { dias_desde_alta: { $dateDiff: { startDate: "$alta_anterior", endDate: "$fecha_admision", unit: "day" } },
            dx_actual: { $first: { $filter: { input: "$diagnosticos", cond: { $eq: ["$$this.tipo", "PRINCIPAL"] } } } }
      } } },
  { $match: { dias_desde_alta: { $lte: 30 } } },
  { $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "_id", as: "p" } },
  { $project: { _id: 0, paciente: { $concat: [{ $first: "$p.nombres" }, " ", { $first: "$p.apellidos" }] },
                anterior: { $concat: ["$hosp_anterior", " (", "$dx_anterior", ")"] },
                reingreso: { $concat: ["$_id", " (", "$dx_actual.cie10", ")"] },
                dias_desde_alta: "$dias_desde_alta", mismo_dx: { $eq: ["$dx_anterior", "$dx_actual.cie10"] } } },
  { $sort: { dias_desde_alta: 1 } }
]).forEach(fila);
const egresos = hce.hospitalizaciones.countDocuments({ ...vigente, estado: "EGRESADA" });
print("(egresos en el periodo: " + egresos + ")");
```

```
mongosh --file scripts/04_consultas_avanzadas.js

==================== A2. Reingresos a 30 dias (ventana por paciente con $setWindowFields) ====================
paciente: Héctor Darío Jaramillo Builes | anterior: HOSP-2026-0026 (E11.1) | reingreso: HOSP-2026-0033 (N39.0) |
    dias_desde_alta: 10 | mismo_dx: false
paciente: Martha Lucía Álvarez Restrepo | anterior: HOSP-2026-0022 (J18.9) | reingreso: HOSP-2026-0025 (J18.9) |
    dias_desde_alta: 11 | mismo_dx: true
paciente: Jorge Eliécer Muñoz Pérez | anterior: HOSP-2026-0009 (J44.1) | reingreso: HOSP-2026-0013 (J44.1) | dias_desde_alta: 12
     | mismo_dx: true
paciente: Rosa Elvira Gómez de Zapata | anterior: HOSP-2026-0002 (I50.0) | reingreso: HOSP-2026-0007 (I50.0) | dias_desde_alta:
    15 | mismo_dx: true
(egresos en el periodo: 29)
```

Figura 11. Resultado real de A2.

Qué revela: 4 reingresos sobre 29 egresos (13,8 %); 3 por el mismo diagnóstico (neumonía, EPOC, falla cardiaca), un patrón típico de pacientes crónicos que requieren seguimiento tras el alta. $setWindowFields compara cada ingreso con el anterior del mismo paciente sin necesidad de un self-join.

A3. Ocupación de camas por servicio

Pregunta: ¿Cuánta capacidad queda en cada servicio y quién ocupa cada cama?
Etapas: $lookup con sub-pipeline (solo hospitalizaciones activas) → $group con $cond → $set (porcentaje) → $sort → $project
Responsable: Henry Taborda

```
titulo("A3. Ocupacion de camas por servicio (estado actual)");
hce.camas.aggregate([
  { $lookup: { from: "hospitalizaciones", localField: "hospitalizacion_id", foreignField: "_id", as: "h",
               pipeline: [{ $match: { estado: "ACTIVA", ...vigente } },
                          { $project: { paciente_id: 1, dias: { $dateDiff: { startDate: "$fecha_admision", endDate:
      CORTE, unit: "day" } } } }] } },
  { $group: { _id: "$servicio", total: { $sum: 1 },
              ocupadas: { $sum: { $cond: [{ $eq: ["$estado", "OCUPADA"] }, 1, 0] } },
              mantenimiento: { $sum: { $cond: [{ $eq: ["$estado", "MANTENIMIENTO"] }, 1, 0] } },
              pacientes: { $push: { $cond: [{ $gt: [{ $size: "$h" }, 0] },
                { $concat: ["$_id", ":", { $first: "$h.paciente_id" }, "(", { $toString: { $first: "$h.dias" } },
      "d)"] }, "$$REMOVE"] } } } },
  { $set: { ocupacion_pct: { $round: [{ $multiply: [{ $divide: ["$ocupadas", { $subtract: ["$total",
      "$mantenimiento"] }] }, 100] }, 1] } } },
  { $sort: { ocupacion_pct: -1 } },
  { $project: { _id: 0, servicio: "$_id", total: "$total", ocupadas: "$ocupadas", mantenimiento: "$mantenimiento",
      ocupacion_pct: "$ocupacion_pct", pacientes: "$pacientes" } }
]).forEach(fila);
```

```
mongosh --file scripts/04_consultas_avanzadas.js

==================== A3. Ocupacion de camas por servicio (estado actual) ====================
servicio: MEDICINA_INTERNA | total: 12 | ocupadas: 6 | mantenimiento: 1 | ocupacion_pct: 54.5 | pacientes: [MI-201:PAC-022(7d),
    MI-202:PAC-007(5d), MI-203:PAC-002(4d), MI-204:PAC-006(3d), MI-205:PAC-024(1d), MI-206:PAC-025(0d)]
servicio: PEDIATRIA | total: 6 | ocupadas: 1 | mantenimiento: 0 | ocupacion_pct: 16.7 | pacientes: [PE-101:PAC-020(6d)]
servicio: CIRUGIA | total: 6 | ocupadas: 0 | mantenimiento: 0 | ocupacion_pct: 0 | pacientes: []
servicio: UCI | total: 6 | ocupadas: 0 | mantenimiento: 0 | ocupacion_pct: 0 | pacientes: []
```

Figura 12. Resultado real de A3.

Qué revela: Medicina Interna al 54,5 % (6 de 11 camas útiles; una en mantenimiento no cuenta) con el paciente y los días de estancia de cada cama. Parte de la colección camas, lo que muestra la navegación inversa de la referencia.

A4. Uso de antibióticos (ATC J01) y días de terapia

Pregunta: ¿Qué antibióticos se usan, para qué diagnósticos y durante cuánto tiempo?
Etapas: $match → $set (dx principal) → $unwind órdenes → $lookup catálogo → $match ($regex ^J01) → $group ($dateDiff) → $sort → $project
Responsable: Henry Taborda

```
titulo("A4. Antibioticos (ATC J01): ordenes y dias de terapia por diagnostico principal");
hce.hospitalizaciones.aggregate([
  { $match: vigente },
  { $set: { dx: { $first: { $filter: { input: "$diagnosticos", cond: { $eq: ["$$this.tipo", "PRINCIPAL"] } } } } } },
  { $unwind: "$ordenes_medicamentos" },
  { $lookup: { from: "catalogo_medicamentos", localField: "ordenes_medicamentos.medicamento_id", foreignField: "_id",
      as: "m" } },
  { $match: { "m.atc": { $regex: "^J01" } } },
  { $group: { _id: { antibiotico: { $first: "$m.nombre_generico" }, dx: "$dx.cie10" }, ordenes: { $sum: 1 },
              dias_terapia: { $sum: { $dateDiff: { startDate: "$ordenes_medicamentos.fecha_inicio",
                endDate: { $ifNull: ["$ordenes_medicamentos.fecha_fin", CORTE] }, unit: "day" } } },
              suspendidas: { $sum: { $cond: [{ $eq: ["$ordenes_medicamentos.estado", "SUSPENDIDA"] }, 1, 0] } } } },
  { $sort: { dias_terapia: -1 } },
  { $project: { _id: 0, antibiotico: "$_id.antibiotico", dx: "$_id.dx", ordenes: "$ordenes", dias_terapia:
      "$dias_terapia",
                promedio_dias: { $round: [{ $divide: ["$dias_terapia", "$ordenes"] }, 1] }, suspendidas:
      "$suspendidas" } }
]).forEach(fila);
```

```
mongosh --file scripts/04_consultas_avanzadas.js

==================== A4. Antibioticos (ATC J01): ordenes y dias de terapia por diagnostico principal ====================
antibiotico: Ceftriaxona | dx: J18.9 | ordenes: 4 | dias_terapia: 23 | promedio_dias: 5.8 | suspendidas: 0
antibiotico: Azitromicina | dx: J18.9 | ordenes: 4 | dias_terapia: 23 | promedio_dias: 5.8 | suspendidas: 0
antibiotico: Azitromicina | dx: J44.1 | ordenes: 4 | dias_terapia: 22 | promedio_dias: 5.5 | suspendidas: 0
antibiotico: Ampicilina/sulbactam | dx: K35.8 | ordenes: 3 | dias_terapia: 7 | promedio_dias: 2.3 | suspendidas: 0
antibiotico: Ceftriaxona | dx: N39.0 | ordenes: 3 | dias_terapia: 7 | promedio_dias: 2.3 | suspendidas: 0
antibiotico: Trimetoprim/sulfametoxazol | dx: N39.0 | ordenes: 1 | dias_terapia: 1 | promedio_dias: 1 | suspendidas: 0
```

Figura 13. Resultado real de A4.

Qué revela: La neumonía y la EPOC concentran el uso de ceftriaxona y azitromicina (≈ 23 días de terapia cada una). Es el indicador base de un programa de uso racional de antibióticos.

A5. Alerta temprana de deterioro (NEWS2 simplificado)

Pregunta: ¿Qué paciente hospitalizado necesita valoración urgente ahora?
Etapas: $match → $lookup (edad) → $set ($sortArray + $last: última toma) → $match (≥ 16 años) → $set ($switch por parámetro) → $set (suma y riesgo) → $sort → $project
Responsable: Henry Taborda

```
titulo("A5. NEWS2 simplificado con la ultima toma de signos vitales (pacientes activos >= 16 anos)");
const rango = (campo, tramos) => ({ $switch: { branches: tramos.map(([cond, pts]) => ({ case: cond(campo), then: pts
      })), default: 0 } });
hce.hospitalizaciones.aggregate([
  { $match: { ...vigente, estado: "ACTIVA" } },
  { $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "_id", as: "p",
               pipeline: [{ $project: { nac: "$fecha_nacimiento", nombre: { $concat: ["$nombres", " ", "$apellidos"]
      } } }] } },
  { $set: { edad: { $dateDiff: { startDate: { $first: "$p.nac" }, endDate: CORTE, unit: "year" } },
            v: { $last: { $sortArray: { input: "$signos_vitales", sortBy: { fecha: 1 } } } } } },
  { $match: { edad: { $gte: 16 } } },
  { $set: { pts: {
      fr:   rango("$v.fr", [[c => ({ $lte: [c, 8] }), 3], [c => ({ $lte: [c, 11] }), 1], [c => ({ $lte: [c, 20] }),
      0], [c => ({ $lte: [c, 24] }), 2], [c => ({ $gte: [c, 25] }), 3]]),
      spo2: rango("$v.spo2", [[c => ({ $lte: [c, 91] }), 3], [c => ({ $lte: [c, 93] }), 2], [c => ({ $lte: [c, 95]
      }), 1]]),
      tas:  rango("$v.ta_sistolica", [[c => ({ $lte: [c, 90] }), 3], [c => ({ $lte: [c, 100] }), 2], [c => ({ $lte:
      [c, 110] }), 1], [c => ({ $gte: [c, 220] }), 3]]),
      fc:   rango("$v.fc", [[c => ({ $lte: [c, 40] }), 3], [c => ({ $lte: [c, 50] }), 1], [c => ({ $lte: [c, 90] }),
      0], [c => ({ $lte: [c, 110] }), 1], [c => ({ $lte: [c, 130] }), 2], [c => ({ $gte: [c, 131] }), 3]]),
      temp: rango("$v.temperatura", [[c => ({ $lte: [c, 35] }), 3], [c => ({ $lte: [c, 36] }), 1], [c => ({ $lte: [c,
      38] }), 0], [c => ({ $lte: [c, 39] }), 1], [c => ({ $gt: [c, 39] }), 2]]) } } },
  { $set: { news2: { $add: ["$pts.fr", "$pts.spo2", "$pts.tas", "$pts.fc", "$pts.temp"] },
            max_parametro: { $max: ["$pts.fr", "$pts.spo2", "$pts.tas", "$pts.fc", "$pts.temp"] } } },
  { $set: { riesgo: { $switch: { branches: [
      { case: { $gte: ["$news2", 7] }, then: "ALTO - respuesta de emergencia" },
      { case: { $or: [{ $gte: ["$news2", 5] }, { $eq: ["$max_parametro", 3] }] }, then: "MEDIO - valoracion urgente"
      } ],
      default: "BAJO - vigilancia de rutina" } } } },
  { $sort: { news2: -1 } },
  { $project: { _id: 0, hosp: "$_id", paciente: { $first: "$p.nombre" }, edad: "$edad", cama: "$cama", news2:
      "$news2", riesgo: "$riesgo",
                ultima_toma: { $dateToString: { date: "$v.fecha", format: "%d-%m %H:%M" } },
                signos: { $concat: ["FC ", { $toString: "$v.fc" }, " FR ", { $toString: "$v.fr" }, " TAS ", {
      $toString: "$v.ta_sistolica" },
                                   " SpO2 ", { $toString: "$v.spo2" }, " T ", { $toString: "$v.temperatura" }] } } }
]).forEach(fila);
```

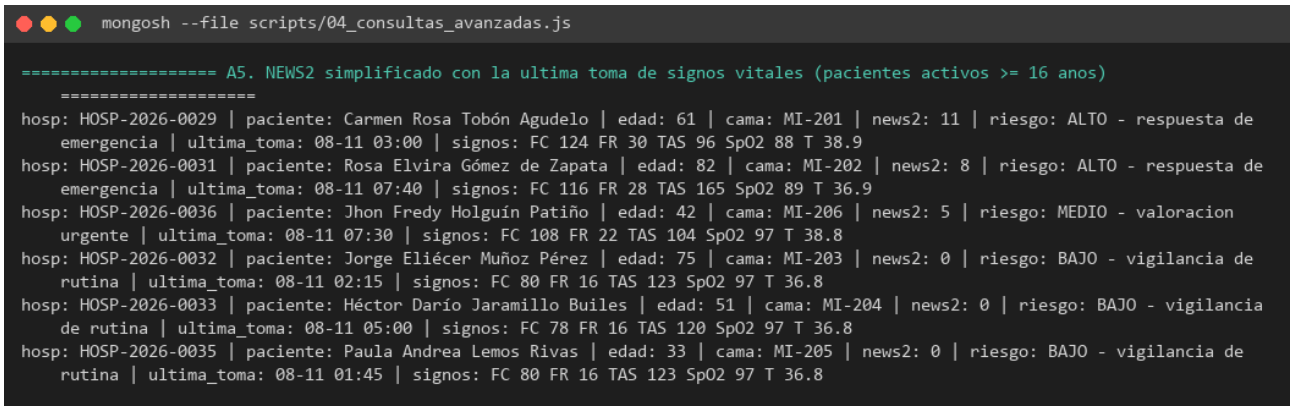


Figura 14. Resultado real de A5.

Qué revela: Dos pacientes en riesgo ALTO (NEWS2 11 y 8: neumonía con SpO_2 88 % y falla cardiaca con SpO_2 89 %) y uno MEDIO. NEWS2 solo es válido en mayores de 16 años, por eso la niña con dengue se excluye; se omiten conciencia y oxígeno suplementario.

A6. Conflictos alergia–medicamento

Pregunta: ¿A quién se le ordenó un medicamento de una familia a la que es alérgico?
Etapas: $match → $lookup (solo pacientes con alergias) → $match → $unwind → $lookup catálogo → $set ($map alergia→familia) → $match ($expr $in) → $project → $sort
Responsable: Henry Taborda

```
titulo("A6. Conflictos alergia-medicamento (cruce paciente x orden x catalogo)");
const FAMILIA = { "Penicilina": "BETALACTAMICO", "AINEs": "AINE", "Sulfas": "SULFONAMIDA", "Quinolonas": "QUINOLONA"
      };
hce.hospitalizaciones.aggregate([
  { $match: vigente },
  { $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "_id", as: "p",
               pipeline: [{ $match: { "alergias.0": { $exists: true } } }] } },     // solo pacientes con alergias
  { $match: { "p.0": { $exists: true } } },
  { $unwind: "$ordenes_medicamentos" },
  { $lookup: { from: "catalogo_medicamentos", localField: "ordenes_medicamentos.medicamento_id", foreignField: "_id",
      as: "m" } },
  { $set: { familia_med: { $first: "$m.familia_alergia" },
            familias_alergicas: { $map: { input: { $first: "$p.alergias" }, as: "a",
              in: { $getField: { field: "$$a", input: { $literal: FAMILIA } } } } } } },
  { $match: { $expr: { $in: ["$familia_med", "$familias_alergicas"] } } },
  { $project: { _id: 0, hosp: "$_id", estado_hosp: "$estado", paciente: "$paciente_id", alergias: { $first:
      "$p.alergias" },
                medicamento: "$ordenes_medicamentos.nombre", familia: "$familia_med", orden:
      "$ordenes_medicamentos.estado" } },
  { $sort: { orden: 1, hosp: 1 } }
]).forEach(fila);
```

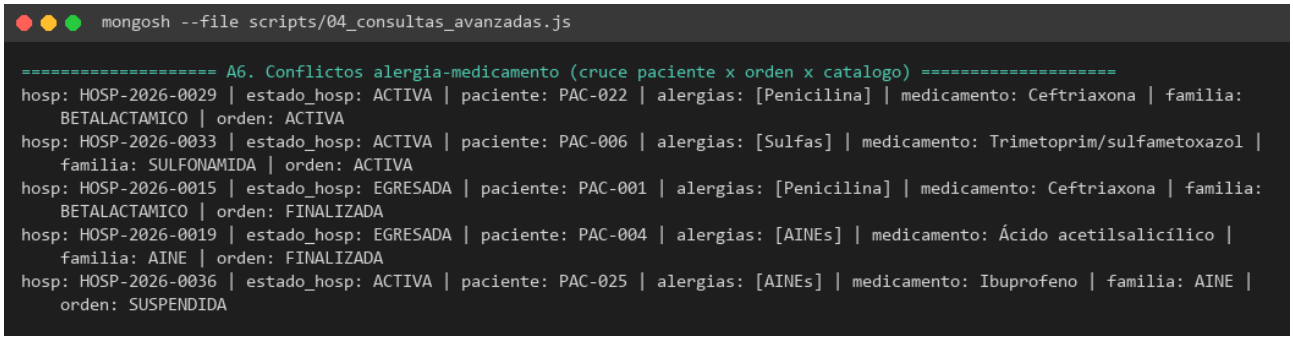


Figura 15. Resultado real de A6.

Qué revela: Dos órdenes ACTIVAS requieren revisión: trimetoprim/sulfametoxazol en un paciente alérgico a sulfas y ceftriaxona en una paciente alérgica a penicilina (reactividad cruzada baja, pero debe confirmarse). El ibuprofeno de U3 aparece SUSPENDIDO: la consulta confirma que el CRUD corrigió el riesgo.

A7. Estancia y desenlace por diagnóstico

Pregunta: ¿Qué diagnósticos consumen más días-cama y cuáles tienen peor desenlace?
Etapas: $match → $set ($dateDiff) → $group ($sum, $avg, $median, $max, $cond) → $lookup catálogo → $sort → $project
Responsable: Henry Taborda

```
titulo("A7. Estancia (dias) y desenlace por diagnostico principal - hospitalizaciones egresadas");
hce.hospitalizaciones.aggregate([
  { $match: { ...vigente, estado: "EGRESADA" } },
  { $set: { dias: { $dateDiff: { startDate: "$fecha_admision", endDate: "$alta.fecha", unit: "day" } } } },
  { $group: { _id: "$alta.diagnostico_egreso", egresos: { $sum: 1 }, dias_cama: { $sum: "$dias" },
              promedio: { $avg: "$dias" }, mediana: { $median: { input: "$dias", method: "approximate" } }, maximo: {
      $max: "$dias" },
              fallecidos: { $sum: { $cond: [{ $eq: ["$alta.condicion", "FALLECIDO"] }, 1, 0] } } } },
  { $lookup: { from: "catalogo_cie10", localField: "_id", foreignField: "_id", as: "c" } },
  { $sort: { dias_cama: -1 } },
  { $project: { _id: 0, cie10: "$_id", diagnostico: { $first: "$c.descripcion" }, egresos: "$egresos", dias_cama:
      "$dias_cama",
               promedio: { $round: ["$promedio", 1] }, mediana: "$mediana", maximo: "$maximo", fallecidos:
      "$fallecidos" } }
]).forEach(fila);
```

```
mongosh --file scripts/04_consultas_avanzadas.js

==================== A7. Estancia (dias) y desenlace por diagnostico principal - hospitalizaciones egresadas
    ====================
cie10: J44.1 | diagnostico: EPOC con exacerbación aguda | egresos: 3 | dias_cama: 18 | promedio: 6 | mediana: 6 | maximo: 7 |
    fallecidos: 0
cie10: A90 | diagnostico: Dengue sin signos de alarma | egresos: 4 | dias_cama: 17 | promedio: 4.2 | mediana: 4 | maximo: 5 |
    fallecidos: 0
cie10: J18.9 | diagnostico: Neumonía, no especificada | egresos: 3 | dias_cama: 16 | promedio: 5.3 | mediana: 5 | maximo: 6 |
    fallecidos: 0
cie10: A91 | diagnostico: Dengue grave | egresos: 2 | dias_cama: 14 | promedio: 7 | mediana: 7 | maximo: 7 | fallecidos: 0
cie10: I50.0 | diagnostico: Insuficiencia cardíaca congestiva | egresos: 2 | dias_cama: 13 | promedio: 6.5 | mediana: 6 |
    maximo: 7 | fallecidos: 0
cie10: E11.1 | diagnostico: Diabetes mellitus tipo 2 con cetoacidosis | egresos: 2 | dias_cama: 9 | promedio: 4.5 | mediana: 4 |
     maximo: 5 | fallecidos: 0
cie10: J21.9 | diagnostico: Bronquiolitis aguda, no especificada | egresos: 2 | dias_cama: 9 | promedio: 4.5 | mediana: 4 |
    maximo: 5 | fallecidos: 0
cie10: A09 | diagnostico: Diarrea y gastroenteritis de presunto origen infeccioso | egresos: 3 | dias_cama: 9 | promedio: 3 |
    mediana: 3 | maximo: 3 | fallecidos: 0
cie10: I21.9 | diagnostico: Infarto agudo de miocardio, sin otra especificación | egresos: 2 | dias_cama: 9 | promedio: 4.5 |
    mediana: 3 | maximo: 6 | fallecidos: 1
cie10: K35.8 | diagnostico: Apendicitis aguda, otra y la no especificada | egresos: 3 | dias_cama: 7 | promedio: 2.3 | mediana:
    2 | maximo: 3 | fallecidos: 0
cie10: I10 | diagnostico: Hipertensión esencial (primaria) | egresos: 2 | dias_cama: 7 | promedio: 3.5 | mediana: 3 | maximo: 4
    | fallecidos: 0
cie10: N39.0 | diagnostico: Infección de vías urinarias, sitio no especificado | egresos: 1 | dias_cama: 3 | promedio: 3 |
    mediana: 3 | maximo: 3 | fallecidos: 0
```

Figura 16. Resultado real de A7.

Qué revela: La EPOC exacerbada es la que más días-cama consume (18); el dengue grave tiene la estancia promedio más alta (7 días) y el infarto registra el único fallecimiento del periodo.

O1. Optimización verificada con explain()

Pregunta: casos de dengue (A90/A91) en Urabá con el nombre del paciente. Se compararon tres versiones del mismo pipeline con `explain('executionStats')`. La IA sugirió poner el `$match` antes del `$lookup`; al medir, se encontró que MongoDB **ya mueve solo** la parte del filtro que no depende del `$lookup` (V1 y V2 examinan lo mismo y usan el índice). El ahorro real aparece frente a un filtro que solo existe después del `$lookup` (V0): de 72 a 16 documentos examinados (−78 %), de COLLSCAN a IXSCAN, con el mismo resultado.

```
const sinIndice = [      // filtro que solo existe despues del $lookup: no puede usar indices
  { $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "_id", as: "p" } },
  { $match: { "p.contacto.subregion": "Uraba" } },
  { $set: { dengue: { $in: [{ $literal: true }, { $map: { input: "$diagnosticos", in: { $in: ["$$this.cie10", ["A90",
      "A91"]] } } }] } } },
  { $match: { dengue: true } },
  { $project: { _id: 1, paciente: { $first: "$p.nombres" } } }];
const lookupPrimero = [  // escrito "mal": $lookup antes del filtro por diagnostico
  { $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "_id", as: "p" } },
  { $match: { "diagnosticos.cie10": { $in: ["A90", "A91"] }, "p.contacto.subregion": "Uraba" } },
  { $project: { _id: 1, paciente: { $first: "$p.nombres" } } }];
const matchPrimero = [   // escrito "bien": $match indexable primero y $lookup con sub-pipeline minima
  { $match: { "diagnosticos.cie10": { $in: ["A90", "A91"] } } },
  { $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "_id", as: "p",
               pipeline: [{ $match: { "contacto.subregion": "Uraba" } }, { $project: { nombres: 1 } }] } },
  { $match: { "p.0": { $exists: true } } },
  { $project: { _id: 1, paciente: { $first: "$p.nombres" } } }];
```

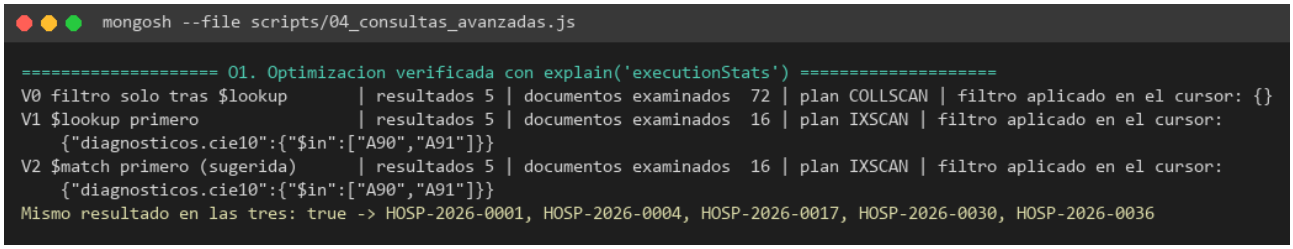


Figura 17. O1: documentos examinados y plan de ejecución de las tres versiones.

5. Integración de herramientas de IA

Se usó **Claude Code** (Anthropic, modelo Claude Opus 5.5) en la terminal, con acceso a los archivos del proyecto y al servidor MongoDB local. La regla fue: **todo lo que la IA propone se ejecuta contra la base real** y solo se acepta lo que la ejecución confirma. La traza completa (prompts y resultados) está en `ia_evidencia/EVIDENCIA_IA.md`.

#	Prompt / situación	Resultado de la IA	Cómo mejoró el trabajo
1	«Hazme esta tarea siguiendo lo que ya hicimos en la asignatura» + enunciado y rúbrica	Leyó la entrega de la Unidad 2 y propuso ampliar ese modelo (camas, auditoría, borrado lógico, familias de alergia, subregión) y generar 35 hospitalizaciones desde perfiles clínicos.	Continuidad con la U2 y datos con volumen para análisis.
2	Error: «Expression not supported in partial index: $not»	Explicó que los índices parciales no admiten $exists:false y propuso la alternativa de un booleano.	Se documentó una limitación real del borrado lógico (sección 1.3).
3	Revisión de una muestra de los datos cargados	Detectó que la niña de 10 años con dengue tenía dosis de adulto y agregó dosis pediátricas por peso.	Pasar el validador no significa que el dato tenga sentido clínico.
4	La salida de C3 decía «código 121 (undefined)»	Leyó errInfo.details.schemaRulesNotSatisfied y ahora lista cada regla incumplida.	Evidencia clara de la validación en el informe y el video.
5	Resultados de $dateDiff impresos como {high, low}	Escribió un formateador de Long y fechas; detectó y revirtió un reemplazo automático que había dañado tres $sort.	Salidas legibles; todo cambio automático se revisa.
6	«Optimiza el pipeline de dengue en Urabá»	Propuso $match antes de $lookup; al medir con explain() corrigió su propia afirmación: el optimizador ya lo hacía.	La consulta O1 explica lo que de verdad pasa dentro del motor.
7	«Propón consultas con valor clínico real»	Diseñó A1–A7 a partir de indicadores hospitalarios y señaló sus límites (NEWS2 solo en adultos).	Consultas orientadas a decisiones, no búsquedas simples.
8	Revisión final del informe	Detectó que U5 asignaba el código de evento 210 también al dengue grave (correcto: 220) y una flecha errónea en el diagrama.	Se corrigieron el script y el diagrama, y se regeneraron todas las evidencias.

5.1 Prompts reutilizables

- «Explícame etapa por etapa qué hace este pipeline y qué documentos salen de cada etapa: [pipeline]»
- «Este comando de mongosh falla con [error]. Explícame la causa y propón la corrección mínima.»
- «Ejecuta explain('executionStats') de este pipeline e indícame si usa índice y cuántos documentos examina.»
- «Compara cómo resolverías la consulta A2 (reingresos) en MongoDB, en SQL y en Neo4j (Cypher).»

Responsabilidad humana: la IA no tuvo acceso a datos reales (todo es sintético). El autor revisó cada script, puede explicarlo y es responsable de las decisiones de diseño. Las dosis, esquemas y puntajes simulados son ilustrativos y no constituyen una guía médica.

6. Reflexión ética sobre la manipulación de datos clínicos

Los datos de la historia clínica son **datos sensibles** (Ley 1581 de 2012, art. 5) y están sometidos a reserva (Ley 23 de 1981, art. 34; Res. 1995 de 1999). La Ley 2015 de 2020 crea la historia clínica electrónica interoperable, lo que significa que un sistema como este, en producción, movería datos de millones de antioqueños entre instituciones. Las decisiones técnicas de este trabajo tienen consecuencias éticas concretas:

6.1 Privacidad: el documento agregado y la reidentificación

- **Riesgo del embebido.** Un documento de hospitalización concentra diagnósticos, evolución, medicamentos y signos vitales: quien lo lee obtiene el cuadro clínico completo. MongoDB controla permisos por colección, no por subdocumento, así que el mínimo privilegio exige *vistas* con proyecciones por rol (enfermería, facturación, epidemiología).
- **Reidentificación por conteos pequeños.** A1 publica casos por subregión: Occidente y Magdalena Medio tienen **un solo caso**. En un municipio pequeño, «el único hospitalizado por dengue de Puerto Berrío» es identificable. Antes de publicar un reporte se deben suprimir o agrupar celdas con menos de 5 casos (criterio de k-anonimato).
- **Minimización aplicada.** El `$lookup` de A1 trae solo la subregión del paciente, no su nombre ni documento. En O1, la versión optimizada proyecta solo el nombre. Pedir menos datos reduce el impacto de una fuga.

6.2 Trazabilidad y conservación

- **Auditoría.** Cada operación del CRUD deja un registro en `auditoria`. Es una colección propia y no un subdocumento para que quien edita una historia no pueda alterar su rastro. En producción debería ser de solo inserción a nivel de permisos (rol sin update/delete) y alimentarse también del *auditLog* del servidor.
- **Borrado lógico vs. definitivo.** La historia clínica debe conservarse por años (Res. 1995/1999, modificada por la Res. 839/2017), por eso un registro erróneo se marca como eliminado, con motivo y responsable, en lugar de destruirse. El borrado físico (D3) solo se usó en un duplicado sin contenido clínico, y el script lo verifica antes de borrar. A la vez, el borrado lógico **no es** supresión: si un titular ejerce su derecho a la supresión cuando proceda, el dato sigue existiendo y debe tratarse con un proceso de anonimización documentado.
- **Correcciones visibles.** U3 no borra la orden de ibuprofeno: la suspende con fecha y motivo. Una historia clínica debe mostrar qué se ordenó y por qué se corrigió.

6.3 Gobernanza e integridad

- **Esquema flexible no significa sin reglas.** Sin validadores, dos IPS podrían guardar la temperatura como texto o en otra unidad y corromper los análisis. Aquí el validador rechazó un paciente con 5 errores (C3).
- **Referencias sin claves foráneas.** MongoDB no impide que `paciente_id` apunte a un paciente inexistente ni que la descripción copiada de un medicamento quede desactualizada. Esto exige procesos de verificación de referencias y un responsable del dato maestro (catálogos CIE-10 y medicamentos).
- **Atomicidad.** La admisión toca dos colecciones (cama y hospitalización). En producción se requiere un replica set y transacciones multidocumento; en este entorno de pruebas se usó un orden de escritura seguro.

6.4 Sesgos y límites de los algoritmos clínicos

- **NEWS2 (A5)** está validado en adultos: aplicarlo a niños daría falsas alarmas o falsa tranquilidad, por eso se excluyen. La versión simplificada omite conciencia y oxígeno suplementario: es un apoyo, no un reemplazo del juicio clínico.
- **Alergias en texto libre (A6).** El cruce depende de que la alergia se escriba exactamente igual («Penicilina», no «PCN»). Una alergia mal digitada produce un falso negativo con riesgo para el paciente. Se recomienda codificarlas con un terminológico estándar. A la vez, alertar por familia completa genera falsos positivos (penicilina–ceftriaxona) y la fatiga de alertas hace que el personal las ignore.
- **Datos sintéticos.** Los perfiles fueron diseñados por el autor; los patrones que revelan las consultas (dengue en Urabá, reingresos de crónicos) son plausibles pero **no son evidencia epidemiológica** y no deben usarse para decisiones reales.

6.5 Prácticas responsables propuestas

Riesgo	Práctica
Acceso excesivo al documento agregado	RBAC con roles por perfil y vistas con proyección; autenticación obligatoria (este entorno local no la tiene y solo escucha en 127.0.0.1).
Fuga de datos	Cifrado en reposo y TLS; cifrado a nivel de campo (Queryable Encryption / CSFLE) para documento y nombre.
Reidentificación en reportes	Supresión de celdas < 5 y seudonimización para analítica.
Alteración del rastro	Auditoría de solo inserción, auditLog del servidor y revisión periódica.
Pérdida o destrucción de historia	Borrado lógico por defecto, respaldos cifrados y política de retención documentada.
Uso de IA con datos clínicos	Nunca enviar datos reales a herramientas de IA externas; usar datos sintéticos como en este trabajo.

7. Distribución del trabajo, repositorio y video

Este trabajo se realizó de forma **individual**, por lo que todas las actividades y consultas estuvieron a cargo del autor, con el apoyo de IA documentado en la sección 5.

Integrante	Rol	Consultas propias
Henry Taborda	Diseño del modelo, CRUD, consultas avanzadas, evidencia de IA, reflexión ética, informe y video	A1, A2, A3, A4, A5, A6, A7, O1

7.1 Repositorio digital

Enlace: https://github.com/HenryStark866/bd2-ae3-u3-hce-hospitalizacion

```
Grupo_AE3_U3_Entrega/
├─ Grupo1_AE3_U3.pdf          <- este informe
├─ README.md                      <- cómo reproducir
├─ scripts/   01_crear_esquema.js  02_carga_datos.js  03_crud.js  04_consultas_avanzadas.js
│             exportar_json.py  generar_capturas.py  generar_diagrama.py
├─ datos_json/                    <- las 7 colecciones exportadas (datos de prueba)
├─ resultados/                    <- salidas reales de mongosh (.txt)
├─ capturas/                      <- diagrama y capturas de ejecución (.png)
├─ ia_evidencia/EVIDENCIA_IA.md   <- prompts, resultados y aporte de la IA
├─ video/GUION_VIDEO.md           <- guion del video (<= 10 min)
└─ informe_fuente/                <- código que genera este PDF y config.json
```

Reproducir: `mongosh --file scripts/01_crear_esquema.js` → `02_carga_datos.js` → `03_crud.js` → `04_consultas_avanzadas.js`. Al volver a ejecutar los cuatro scripts desde cero se obtienen exactamente las mismas salidas que están en `resultados/` (verificado con diff).

7.2 Video demostrativo

Enlace: pendiente

Duración máxima 10 minutos. Muestra la ejecución en vivo de los cuatro scripts: creación del modelo, carga de datos, cada bloque CRUD (con la auditoría al final), las consultas A1–A7 explicando la pregunta clínica y el hallazgo, la optimización O1, la evidencia de IA y la reflexión ética. El guion por minutos está en `video/GUION_VIDEO.md`.

8. Conclusiones

- El modelo documental de la Unidad 2 soportó sin rediseño las operaciones complejas: los arrays embebidos se manipulan con precisión ($push $each, arrayFilters, posicional $, $[], $pull) sin reescribir el documento completo.
- El Aggregation Framework permite análisis clínico real (epidemiología territorial, reingresos con funciones de ventana, puntajes de alerta, seguridad del paciente) directamente en la base, sin exportar datos.
- Las referencias del modelo híbrido tienen costo: cada $lookup es un join que hay que medir; explain() mostró que el índice multikey, más que el orden de las etapas, es lo que reduce el trabajo (−78 % de documentos examinados).
- La flexibilidad de NoSQL debe compensarse con validación, auditoría y borrado lógico: en datos clínicos la integridad y la trazabilidad no son opcionales.
- Este modelo documental complementa el relacional de las unidades anteriores: lo transaccional y maestro (pacientes, catálogos) se beneficia de la integridad relacional, y el episodio clínico variable y de alto volumen, del documento. Esa es la base del diseño híbrido del proyecto integrador.